		Stream
		.filter(...) // reduce data
		.distinct() // remove duplicates
		.map(...) // transform
		.flatMap(...) // flatten if needed
		.sorted(...) // expensive → late
		.peek(...) // debug only
		.collect(...); // terminal operation

Operation	**Use Case**	**One-Line Code Example**
filter()	boolean checks	.filter(x -> x > 10)
filter()	null checks	.filter(Objects::nonNull)
filter()	comparisons	.filter(e -> e.getSalary() >= 50000)
filter()	method calls returning boolean	.filter(User::isActive)
-		
distinct()	rely on equals() / hashCode()	.distinct()
distinct()	immutable objects	.map(String::toUpperCase).distinct()
map()	transform objects	.map(Employee::getName)
map()	create new objects	.map(e -> new EmployeeDTO(e.getId(), e.getName()))
map()	call pure functions	.map(Math::abs)

flatMap()	return Stream	.flatMap(s -> Arrays.stream(s.split(","))))
flatMap()	split data	.flatMap(sentence -> Arrays.stream(sentence.split(" ")))
flatMap()	expand collections	.flatMap(order -> order.getItems().stream())
-		
sorted()	natural ordering	.sorted()
sorted()	custom comparator	.sorted(Comparator.comparing(Employee::getSalary))
sorted()	multi-field compare	.sorted(Comparator.comparing(Employee::getDept).thenComparing(Employee::getSalary))
peek()	logging	.peek(e -> System.out.println(e))
peek()	debugging	.peek(e -> log.debug("Processing: {}", e))
peek()	metrics	.peek(e -> counter.incrementAndGet())
-		
collect()	accumulate data	.collect(Collectors.toList())
collect()	grouping	.collect(Collectors.groupingBy(Employee::getDepartment))
collect()	reduction	.collect(Collectors.summingInt(Employee::getSalary))
collect()	custom collectors	.collect(Collector.of(ArrayList::new, List::add, (a,b)->{a.addAll(b);return a;}))